VOID CTF — Forensics Category Writeup

Author: Akshat (@akshat-xcode) | **Team:** Code Crew | **Category:** Forensics (5/5 Solved) | 1400 / 1400 Points

#	Challenge	Points	Vulnerability / Artifact Type	Flag
1	**I Heard something**	100	Audio Spectrogram (Ultrasonic Band 16-19 kHz)	`VOID{y0u_c4n7_h34r_m3}`
2	**Left Behind**	200	PCAPNG USB HID Keystroke Sniffing (usbmon)	`VOID{h1d_k3ystr0k3s}`
3	**A Sealed Record**	300	Corrupted PNG IHDR Height & CRC-32 Repair	`VOID{crc_r3v34ls_th3_tru3_h31ght}`
4	**Volatile Mira**	300	LiME Linux RAM Dump & Process Arg XOR Decryption	`VOID{f0r3n51c5_15_4n_4r7}`
5	**3000 Files Later**	500	ZIP DOS Epoch (1980) Rule & SHA-256 Signature	`VOID{y0u_ch3ck3d_3000_f1l3s_f0r_th1s_021959f6}`

1. I Heard something

100 Points

> *"Mira left a voice note on her desktop called 'whisper.' It's just a tune. Or is it? Some things are only meant for machines to hear."*

ANALYSIS & SOLUTION

- Provided file: `whisper.wav` (44.1 kHz, 16-bit Mono PCM).
- Hint specifies "meant for machines to hear", pointing to frequencies above normal human hearing range (>15 kHz).
- Computing the Short-Time Fourier Transform (STFT) reveals a synthetic text spectrogram drawn between **16.3 kHz and 19.3 kHz** from $t = 2.0 ext{s}$ to $10.0 ext{s}$.

SOLVE SCRIPT (PYTHON)

```
import scipy.io.wavfile as wavfile, matplotlib.pyplot as plt
sr, data = wavfile.read("whisper.wav")
plt.specgram(data, NFFT=2048, Fs=sr, noverlap=1024, cmap='viridis')
plt.ylim(15000, 20000) # Zoom into ultrasonic band
plt.savefig("flag.png"); plt.show()
```

Flag: `VOID{y0u_c4n7_h34r_m3}`

2. Left Behind

200 Points

> *"Arlo Vance trusts nothing that touches a network... someone left a small device plugged in behind his desk. This file is all it left behind."*

ANALYSIS & SOLUTION

- Provided file: `left_behind.pcapng` (Linktype 220: `LINKTYPE_USB_LINUX_MMAPPED` via Linux `usbmon`).
- USB enumeration shows Device 4 is a Logitech USB Keyboard sending 8-byte HID reports on interrupt endpoint `0x81`.
- Parsed HID usage keycodes; handled modifier bits (Shift) and keystroke edits: Arlo typed `VOID{h1d_`, made a typo `wrong`, pressed Backspace 5 times, and completed `k3ystr0k3s}`.

SOLVE SCRIPT (PYTHON SNIPPET)

```
# Extracting USB HID keystrokes from pcapng EPB packets:
# Dev 4, EP 0x81: payload[0] = modifier, payload[2:8] = keycodes
# 0x2a = Backspace (pops buffer), Shift mapped via HID usage tables
# Result after replay: VOID{h1d_k3ystr0k3s}
```

Flag: `VOID{h1d_k3ystr0k3s}`

3. A Sealed Record

300 Points

> *"the tag was rewritten after sealing. the seal still remembers the original. trust the seal, not the tag."*

ANALYSIS & SOLUTION

- Provided archive: `Artifact-BC34.rar` containing `Artifact-BC34.png`. Tools failed to open due to CRC mismatch in `IHDR`.
- Clue interpretation: "The tag was rewritten" = height altered in `IHDR` (set to $1607 ext{ px}$); "The seal still remembers" = stored CRC-32 checksum (`0x14C4F49F`) remained original.
- Brute-forcing candidate heights from 1 to 10000 against stored CRC gave the true height: **1728 px** (0x000006C0).
- Patching bytes `0x14 - 0x17` revealed 121 cropped rows at the bottom containing the evidence banner.

SOLVE SCRIPT (PYTHON)

```
import struct, zlib
data = bytearray(open("Artifact-BC34.png", "rb").read())
# Brute-force height matching IHDR CRC 0x14C4F49F -> gives height = 1728
data[20:24] = struct.pack(">I", 1728) # Patch 1607 -> 1728
open("repaired.png", "wb").write(data)
```

Flag: `VOID{crc_r3v34ls_th3_tru3_h31ght}`

4. Volatile Mira

300 Points

> *"RAM forgets only when the power goes. Her notes daemon keeps everything encrypted in memory, and she swears the key never sits next to the vault."*

ANALYSIS & SOLUTION

- Provided file: `mem.rar` containing 2 GB Linux LiME memory dump (`mem.lime`).
- Scanned physical pages for daemon scripts; recovered `/opt/mira/heartbeat.py` containing ciphertext: `SEALED = bytes.fromhex("632c7f761e530146035b020402566804003c025c3a0143034d")`
- Clue says "key never sits next to the vault" → checked process command line / history buffers in memory. Found: `python3 /opt/mira/heartbeat.py --session-token 5c62e5140575ac75` .
- Decrypted with repeating-key XOR: $P_i = C_i \oplus ext{Token}_{i \pmod{16}}$.

SOLVE SCRIPT (PYTHON)

```
SEALED = bytes.fromhex("632c7f761e530146035b020402566804003c025c3a0143034d")
KEY = b"5c62e5140575ac75"
flag = bytes([c ^ KEY[i % len(KEY)] for i, c in enumerate(SEALED)])
print(flag.decode()) # VOID{f0r3n51c5_15_4n_4r7}
```

Flag: `VOID{f0r3n51c5_15_4n_4r7}`

5. 3000 Files Later

500 Points

> *"Thousands of flags, every one of them convincing. Note - Before you start grepping: they saw you coming."*

ANALYSIS & SOLUTION

- ZIP archive contains 3,369 files, all with convincing Base64 decoy flags matching `VOID{...}` .
- ZIP comment: "*Ask the one that marks the beginning of an era how to sign. Trust the youngest who bears the mark.*"
- Beginning of era = DOS epoch (`1980-01-01 00:00:00`). File `ae/4a/d9dc427dacb85135.txt` had this timestamp and gave the signing rule: `SHA256("1e7ea419" + body)[:8] == tail` .
- Evaluating 3,369 files found exactly 8 signed candidates. Sorting by timestamp, the **youngest** (`2015-02-04 06:30:32`) is `b1/ab/54b611bcdb191433.txt` .

SOLVE SCRIPT (PYTHON SNIPPET)

```
# Epoch rule: SHA256("1e7ea419" + body)[:8] == tail
# Filter all 3369 files -> 8 match signature rule
# Youngest matching file (2015-02-04): b1/ab/54b611bcdb191433.txt
# Flag: VOID{y0u_ch3ck3d_3000_f1l3s_f0r_th1s_021959f6}
```

Flag: `VOID{y0u_ch3ck3d_3000_f1l3s_f0r_th1s_021959f6}`